SESSION 04 / PRINTABLE GUIDE

Lesson 11: Version and review Fabric notebooks with Git

Session 04 | Follow-on development workflow
Output: A reviewed notebook change in a customer-approved practice branch, with a clear distinction between Git history, workspace state, and execution.

Why this matters

A saved notebook is not a reviewed version, a commit is not a successful run, and merging a pull request does not automatically publish Warehouse data. Git records code/definition changes so the team can explain what changed, who reviewed it, and which version was tested.

Teach this after Notebook 101 and the main workflow. Use VS Code connected authoring as the next module, not as a prerequisite for learning Git concepts.

Mandatory module preflight

The customer and presenter must verify these **before delivery**, during the 3-5-business-day check-in:

- Git integration and the selected provider are available/approved in the actual Fabric cloud/tenant. General Microsoft documentation lists cloud-hosted Azure DevOps and GitHub providers; that does not prove GCC High support or authorize a cross-cloud connection.
- A customer-approved **private practice repository**, feature branch, and isolated training workspace exist. Do not connect this public training-material repository to a customer workspace or push customer content into it.
- The workspace admin has configured Git integration to the intended repository, branch, and folder. Tenant Git switches, capacity, network rules, credentials, workspace permissions, and repository permissions have been approved.
- Each participant/pair has its own branch/workspace assignment and an agreed reviewer. Do not change the branch of a shared delivery/production workspace.
- The exercise notebook is a separately imported copy of the read-only troubleshooting solution, named for the pair, for example `s04_git_pair01`, with the prepared default Lakehouse. Test its read-only run first.

Only a workspace admin can establish the workspace Git connection. The admin selects **Workspace settings -> Git integration**, chooses the supported provider/approved connection, repository, branch, and folder, and reviews initial sync direction. Use a fresh practice target and a deliberate baseline; do not accept an unexplained initial overwrite of existing workspace/repository items. Students use the prepared connection, not copied tokens or elevated admin roles.

If integration is unavailable, the presenter can teach local Git concepts with approved synthetic sources, but must label Fabric commit/sync **blocked**. That is not a successful live integration.

1. Establish the vocabulary and source of truth

Term	What the learner should understand
Repository	Versioned source, not a backup of all Lakehouse data
Branch	A line of development; a Fabric workspace connects to one branch/folder at a time
Working change	An edit not yet committed; identify whether it lives locally or in the workspace
Diff	The actual change to review, including relevant metadata
Commit	A recorded source snapshot with a useful message, not execution evidence
Push / Fabric Commit to Git	Sends local commits / workspace changes to the connected repository through different tools
Pull request	Review/merge workflow in the Git provider, not a notebook run
Update from Git	Applies the connected branch's incoming item definitions to the workspace
Conflict	Competing changes requiring a deliberate resolution, not permission to discard someone's work

Agree the route: for this lesson, **edit in Fabric and commit through the workspace Source control panel**. Do not also edit the same item locally and push a competing definition during the exercise.

2. Understand the two notebook formats

Our handouts ship portable `.ipynb` files for import. Fabric Git integration serializes the **imported workspace item** into its own source representation, for example:

```text
s04_git_pair01.Notebook/
  .platform
  notebook-content.py
  notebook-settings.json    Optional, managed by Fabric
  Resources/               Optional; separately controlled
```

The documented PySpark source format retains metadata, Markdown, and code sections. Fabric reconstructs the notebook on sync. It is not simply our original `.ipynb` uploaded unchanged, and an arbitrary `.py` file is not automatically a deployable Fabric item.

Notebook cell outputs are not included by Fabric's documented Git sync. However, optional resources or files you manually add to Git can contain data or secrets. Inspect the selected changes; do not assume all attached content is safe because notebook outputs are excluded.

Keep required `.platform` and source-format markers. Do not hand-edit Fabric-managed `notebook-settings.json` or `fs-settings.json` to bypass settings. Lakehouse/environment binding metadata can change; review it and verify the target context after sync. Git versioning does not copy the underlying Lakehouse data, ADLS permissions, secrets, or all runtime state.

3. Make and review one small change

1. Confirm the practice workspace's repository, connected branch/folder, and clean baseline. Record the initial commit reference.
2. Open the pair's read-only practice notebook. Change one Markdown description to clarify that it reads the first daily CSV. Add a harmless code cell that prints a short synthetic version marker, such as `training-revision-1`. Do not change source paths, acceptance rules, permissions, or write targets.
3. Run the notebook and confirm the expected read-only behavior. Saving updates the notebook item, not the repository history.
4. Open **Source control**, inspect the pending item and diff, and explain the Markdown/code changes. Review any metadata/resource changes too.
5. Commit only the agreed notebook/dependency changes to the connected feature branch with a specific message, for example `Clarify source scope in read-only notebook practice`.
6. Open the repository in its provider and locate the resulting commit and native notebook definition. Confirm that the intended change, not unrelated workspace items or data, was recorded.

Record commit reference, intended target, run result, and reviewer. A commit reference alone does not establish a successful notebook run.

4. Review through a pull request

Create a pull request from the practice feature branch to the **facilitator's agreed review branch**. Do not guess `main` or bypass branch protection. The reviewer checks:

- The change matches the stated purpose, without unrelated edits.
- Required source metadata remains intact and bindings are understood.
- No outputs/resources contain credentials or unapproved data.
- The actual tested notebook version, identity/context, and run evidence are recorded; local syntax validation is not called a Fabric execution test.

Merge only if the facilitator authorizes it. A merge updates the target Git branch; it does not switch your workspace to that branch, run the notebook, move raw data, refresh a model, or publish Gold.

5. Prove the reverse direction and discuss conflicts

With the practice workspace clean, have the presenter make one reviewed Markdown-only edit to the notebook's source on **that workspace's connected practice branch**, preserving Fabric markers/metadata. Inspect incoming changes in Source control, then use **Update from Git** and confirm the text in the workspace notebook. Verify the default Lakehouse and rerun the read-only test. Do not sync a branch change whose contents/impact you have not reviewed.

For a conflict demonstration, use only a prepared practice item. Compare the two edits, agree the intended result, resolve using the supported tool, and review the complete item again. Preserve work before updating. Do not solve conflicts by force-pushing, blindly discarding changes, deleting the notebook, or replacing resource bindings with guesses.

Optional: Inspect the same branch locally

The presenter supplies the approved URL and branch. These commands are examples for the **customer's separate practice clone**, not instructions to change this training repository or store credentials in shell history:

```bash
git clone --branch "APPROVED_PRACTICE_BRANCH" "APPROVED_REPOSITORY_URL" fabric-notebook-practice
cd fabric-notebook-practice
git status
git --no-pager log -3 --oneline
git --no-pager diff
```

Replace the placeholders before running. An empty local diff after clone is normal; history shows committed changes. With a clean local tree, `git pull --ff-only` can bring in later commits on that branch. Do not use this as a second authoring/publishing route while someone edits the workspace.

Completion evidence

The learner explains the native source format, identifies the connected branch and sync direction, shows the reviewed change/commit/PR, distinguishes execution from versioning, and demonstrates an incoming change without losing work. Record unavailable Git operations as blocked, not completed by screenshots.

Next: VS Code locally connected to Fabric.

Official references, reviewed October 29, 2026: Git overview, connection prerequisites and operations, and notebook source control/bindings.

Source: 11_git_and_fabric_notebooks.md. Companion-material links open the repository originals.